2026 年华北电力大学（保定）新生程序设计大赛题解

A – H　题解册

华北电力大学（保定）

2026

目　录

A　请输入文本 **1**

B　神奇的优惠券 **1**

C　Fn+Del 括号 **2**

D　哥德巴赫猜想 **3**

E　Cutter **4**

F　快看，是一名铁驭 2 **5**

G　Château de cartes **7**

H　星象仪 **8**

A 请输入文本

解题思路

从 1 到 X 循环输出每一届的标题即可。

注意第 $1,2,3$ 届的标题需要分别修改中间部分为 `1-th` 、 `2-nd` 、 `3-rd` 。

复杂度

时间复杂度 $O(X)$ 。

B 神奇的优惠券

解题思路

注意到优惠券只能对一个商品使用，所以只需要枚举每个商品的能少付的价格 $p_i - \left\lfloor \frac{p_i}{k} \right\rfloor$ 取最大值即可。

复杂度

遍历一遍序列即可，时间复杂度 $O(N)$ 。

C Fn+Del 括号

解题思路

合法括号串的充要条件是：任意前缀中左括号的数量都不少于右括号，并且整个字符串中两种括号的数量相等。

从左到右扫描字符串，维护尚未匹配的左括号数量 cnt，以及已经补入的左括号数量 ans，初始均为 0。

- 遇到（ 时，令 cnt 加 1。
- 遇到 ）时，若 $cnt > 0$，就用一个尚未匹配的左括号与它配对，令 cnt 减 1；否则在这个右括号前插入一个（，令 ans 加 1，此时 cnt 仍为 0。

扫描结束后，还剩下 cnt 个左括号没有匹配，在末尾补入 cnt 个右括号即可，答案为 $ans+cnt$。

正确性证明

下面说明这样得到的插入次数最少。令 s_i 表示原串前 i 个字符中左括号数量减去右括号数量，并令 $s_0 = 0$。设

$$L = \max\left(0, -\min_{0 \le i \le n} s_i\right).$$

对于左括号最不足的那个前缀，至少需要插入 L 个左括号才能使其合法，插入右括号只会让这一前缀更加不合法。因此，任意合法方案插入的左括号数量 x 都满足 $x \ge L$。若它插入了 y 个右括号，由最终两种括号数量相等可知

$$y = s_n + x,$$

所以总插入次数满足

$$x + y = s_n + 2x \ge s_n + 2L.$$

上述扫描恰好在前缀的左右括号数量差刷新负的最小值时补入左括号，共补入 L 个。扫描结束时 $cnt = s_n + L$，最终总插入次数为 $s_n + 2L$，达到下界，因此最优。

复杂度

时间复杂度 $O(n)$，除输入字符串外只需 $O(1)$ 的辅助空间。

D 哥德巴赫猜想

筛法与答案预处理

需要反复判断一个数是否为素数，可以先用埃氏筛预处理素数表，再枚举每个偶数的分解。

令 $U = 10^6$。将 $0, 1$ 标记为非素数，其余数先标记为素数。依次枚举 $p = 2, 3, \ldots, \lfloor\sqrt{U}\rfloor$，如果 p 仍被标记为素数，就将 $p^2, p^2 + p, p^2 + 2p, \ldots$ 中不超过 U 的数标记为非素数。从 p^2 开始即可，因为更小的合数倍数已经被更小的素因子筛掉了。筛完后，再按递增顺序收集所有素数。

由于询问数量较多，可以直接预处理每个偶数的答案。对于 $n = 4, 6, \ldots, U$，依次枚举不超过 $n/2$ 的素数 p，检查 $n - p$ 是否为素数。第一次检查成功时，记录

$$ans_n = p,$$

并结束这个 n 的枚举。回答询问时，输出 ans_n 和 $n - ans_n$ 即可。特别地，$n = 4$ 时得到 $2 + 2$，因此枚举不能漏掉素数 2。

正确性证明

任何满足题意的分解都必须有 a 为素数且 $a \le n/2$，所以枚举范围包含所有可能的较小加数。素数按递增顺序枚举，第一次成功的 p 就是最小的 a；同时 $n-p$ 已经通过素数检查，且 $p \le n-p$，因此记录的分解合法并满足最小性要求。

有限范围验证

这里不能直接把哥德巴赫猜想当作对任意范围成立的定理。不过，本题只涉及一个有限范围，可以完整枚举验证：对于所有 $4 \le n \le 10^6$ 的偶数，都能找到分解；其中最小加数的最大值为 523，在 $n = 503222$ 时取得。523 是第 99 个素数，因此本题范围内，每个偶数至多检查 99 个素数就能找到答案。这是对本题有限范围的穷举结果，不是对猜想的一般证明。

复杂度

令 L 表示预处理一个偶数时最多检查的素数个数。埃氏筛的时间复杂度为 $O(U \log\log U)$，答案预处理的时间复杂度为 $O(UL)$，每次询问只需 $O(1)$ 时间。因此总时间复杂度为 $O(U \log\log U + UL + T)$，空间复杂度为 $O(U)$。在本题范围内 $L = 99$，实际预处理全部偶数共需进行 3604369 次候选素数检查，可以满足数据范围。

E Cutter

问题转化

直接构造每一个循环移位并比较前 m 个字符，需要 $O(nm)$ 时间，无法满足全部数据。

把字符串 A 的前 $m-1$ 个字符接到 A 的末尾，得到长度为 $n+m-1$ 的字符串 S。当 $m=1$ 时，不需要追加字符。以下字符串下标均从 0 开始。

对于每个 $0 \le k < n$，循环移位 A_k 的前 m 个字符恰好就是

$$S_k S_{k+1} \cdots S_{k+m-1}.$$

因此，问题转化为统计模式串 B 在 S 中出现的次数。由于 S 长度恰好为 $n+m-1$，长度为 m 的子串起点恰好是 $0, 1, \ldots, n-1$，不会漏掉或多算任何一个循环移位。不同起点即使对应相同的字符串，也会分别计数。

KMP 匹配

使用 KMP 算法即可在线性时间内完成匹配。

先为 B 求前缀函数 π，其中 π_i 表示 $B_0 B_1 \cdots B_i$ 的最长相等真前缀与真后缀的长度，“真”表示不能取整个字符串本身。令 $\pi_0 = 0$，对于每个 $i = 1, 2, \ldots, m-1$：

- 先令 $j = \pi_{i-1}$。
- 当 $j > 0$ 且 $B_i \ne B_j$ 时，令 $j = \pi_{j-1}$，尝试更短的相等前后缀。
- 若 $B_i = B_j$，令 j 加 1，最后令 $\pi_i = j$。

然后从左到右扫描 S，维护已匹配的 B 的前缀长度 j，初始为 0。对于当前字符 c，当 $j > 0$ 且 $c \ne B_j$ 时，不断令 $j = \pi_{j-1}$；若 $c = B_j$，就令 j 加 1。当 $j = m$ 时，说明找到了一个完整匹配，答案加 1，随后令

$$j = \pi_{m-1},$$

继续扫描。

正确性与重叠匹配

失配时，已经匹配的部分若还能作为下一次匹配的开头，就必须是这部分字符串的某个相等前后缀。前缀函数恰好依次给出这些候选长度，所以回退不会遗漏匹配。找到完整匹配后也应保留其最长相等前后缀，不能直接跳过整个匹配区间，否则会漏掉重叠出现。例如 $A = \texttt{aaaa}$、$B = \texttt{aaa}$ 时，四个循环移位都符合要求，答案为 4。

复杂度

求前缀函数和匹配时，j 的增加次数都是线性的，而每次回退都会使 j 减小，因此回退总次数也是线性的。总时间复杂度为 $O(n+m)$，存储字符串和前缀函数的空间复杂度为 $O(n+m)$。也可以扫描时通过 $A_{i \bmod n}$ 获取 S_i，避免显式构造 S。

F 快看，是一名铁驭 2

路线的单调性证明

先说明为什么只需考虑从左向右走的路线。

将每个“时间层、点位”看成一个顶点，两层同号点位之间连一条穿越边，同层相邻点位之间连一条行走边。进入一个顶点需要清扫该点，穿越还需要支付 k。所有代价均非负。

如果一条路线重复经过同一个顶点，可以删去两次经过之间的整段绕行。删除后仍是一条连通的路线，需要清扫的顶点集合只会缩小，穿越次数也不会增加；即使某个点原先在绕行时被清扫，删去绕行后也可以等到真正进入它时再清扫。因此，删去绕行不会增加总代价，至少存在一条最优路线不重复经过任何顶点。

对于任意 $0 \le i \le n$，把编号不超过 i 的点位放在左侧，其余点位放在右侧。连接两侧的边只有现在层、过去层各自从 i 到 $i+1$ 的两条边。起点在左侧，终点在右侧，所以路线跨越这条分界线的次数必须为奇数。不重复顶点的路线不会重复使用边，最多只能跨越两次，因此恰好跨越一次，且方向是从左向右。对每条分界线都成立，故这样的最优路线不会向左走。

动态规划

现在可以按点位编号进行动态规划。令 f_i, g_i 分别表示只向右行走、已经处理完点位 i 上可能发生的穿越后，停在现在层、过去层点位 i 的最小代价。当前停留的点位已经安全。

在点位 0，两层危险度均为 0，所以初始状态为

$$f_0 = 0, \qquad g_0 = k.$$

其中 g_0 对应出发时立即穿越到过去层。令 $a_{n+1} = b_{n+1} = 0$，将终点也纳入转移。

对于每个 $i = 1, 2, \ldots, n+1$，先考虑从上一点位直接走到当前点位、尚未在当前点位穿越的代价：

$$u = f_{i-1} + a_i, \qquad v = g_{i-1} + b_i.$$

随后可以选择不穿越，或者在当前点位穿越一次：

$$\begin{aligned} f_i &= \min\left(u,\ v + k + a_i\right), \\ g_i &= \min\left(v,\ u + k + b_i\right). \end{aligned}$$

例如，由过去层到达现在层点位 i 时，v 已经包含清扫过去层点位 i 的代价 b_i，还需要支付穿越代价 k，以及现在层点位 i 的清扫代价 a_i。两层危险度相互独立，不能漏掉目标层的清扫费用。一次穿越意味着当前编号的两个点位都需要安全。

F 快看，是一名铁驭 2（续）

转移的正确性

在同一个点位连续穿越两次会回到原来的层，只会增加非负代价，因此无需考虑。向右行走的路线第一次到达点位 i 时，只可能来自同层的点位 $i-1$，随后也只需选择是否穿越一次，所以上述转移已经覆盖全部可能。结合初始状态，逐点取最小值即可得到最优答案。

答案与实现细节

最终要求到达现在层，因此答案为 f_{n+1}。将终点纳入转移，也就包含了先到达过去层终点、再支付 k 返回现在层的情况。计算 f_i, g_i 时，应先用上一轮的两个状态求出 u, v，再更新当前状态，避免使用已经被覆盖的值。

总代价可能超过 32 位整数范围。例如，始终留在现在层的代价就可能达到 10^{15}，动态规划中的代价应使用 64 位整数保存。

复杂度

时间复杂度 $O(n)$。动态规划本身只需 $O(1)$ 的辅助空间；由于输入先给出整个 a 数组、再给出整个 b 数组，实际实现可以保存 a，读入 b_i 时进行转移，总空间复杂度为 $O(n)$。

G Château de cartes

消去共同倍数

第 i 次攻击前，剩余纸牌屋的高度与本次攻击力都已经乘上了相同的正整数

$$C_i = \prod_{t=1}^{i-1} t = (i-1)!.$$

其中 $i = 1$ 时空乘积为 1。因此，对于尚未被摧毁的第 j 座纸牌屋，判断条件可以化简为

$$\left\lfloor \frac{C_i H_j}{C_i M_i} \right\rfloor = \left\lfloor \frac{H_j}{M_i} \right\rfloor = 0 \quad \Longleftrightarrow \quad H_j < M_i.$$

所以所有倍增操作都可以忽略，直接使用输入的初始高度与攻击力比较即可，也不需要计算可能极大的阶乘。注意条件是严格小于，高度恰好等于攻击力时不会被摧毁。

排序与双指针

将所有高度从小到大排序为 $h_1, h_2, \ldots, h_N$，维护已经被摧毁的纸牌屋数量 p，初始为 0。已经被摧毁的纸牌屋始终构成排序后数组的一个前缀。

按照原顺序处理每次攻击 M_i：先记下 $old = p$，然后只要 $p < N$ 且 $h_{p+1} < M_i$，就令 p 加 1。本次新摧毁的数量为

$$p - old.$$

正确性证明

因为之前被摧毁的都是最矮的一段，而本次满足 $h_j < M_i$ 的纸牌屋也构成一个前缀，所以两者合并后仍然是一个前缀。指针跳过之前被摧毁的部分，只向右经过本次新满足条件的纸牌屋，故差值正好是本次新摧毁的数量。若本次攻击力较小，指针不会移动，输出 0，不需要恢复任何已经消失的纸牌屋。

复杂度

排序需要 $O(N \log N)$ 时间。虽然每次攻击中可能有多次指针移动，但整个过程中 p 最多增加 N 次，因此处理所有攻击只需 $O(N + Q)$ 时间。总时间复杂度为 $O(N \log N + Q)$，空间复杂度为 $O(N)$；攻击力可以按输入顺序逐个读取并处理。

H 星象仪

关键结论

直接模拟一次校准，需要计算一个共有 $n+(n-1)+\cdots+1$ 个数的三角形数阵，时间复杂度为 $O(n^2)$。但 k 可以达到 10^{18}，不能逐次模拟。

关键结论是：**对任意初始序列、任意模数 M，连续进行三次校准都会回到初始序列模 M 的结果**。因此只需考虑 $k \bmod 3$，赛时你只需要考虑到这里就可以解决这道题目。

三次校准还原的完整证明

下面完整证明这个结论。所有等式均在模 M 的意义下成立，所有数都取 0 到 $M-1$ 之间的代表元。令 $d=n-1$，将初始序列生成的数阵记作 x，行号、列号均从 0 开始，即

$$x_{0,j} = a_{j+1} \bmod M \qquad (0 \le j \le d),$$

$$x_{r+1,c} = -x_{r,c} - x_{r,c+1} \qquad (0 \le r < d,\ 0 \le c < d-r).$$

数阵中所有合法下标满足 $r \ge 0$、$c \ge 0$、$r+c \le d$。递推式也可以写成

$$x_{r,c} + x_{r,c+1} + x_{r+1,c} = 0.$$

一次校准得到的新序列，就是右侧边界

$$x_{0,d},\ x_{1,d-1},\ \ldots,\ x_{d,0}.$$

以这一新序列为第一行生成的数阵记作 y。我们先证明，对于每个合法下标 (r,c)，都有

$$y_{r,c} = x_{c,d-r-c}.$$

当 $r=0$ 时，根据新序列的定义，恰有

$$y_{0,c} = x_{c,d-c},$$

所以第一行满足该式。

H 星象仪（续）

三次校准还原的完整证明（续）

假设第 r 行满足该式，考虑下一行任意一个位置 $0 \le c < d - r$。根据递推规则，

$$\begin{aligned} y_{r+1,c} &= -y_{r,c} - y_{r,c+1} \\ &= -x_{c,d-r-c} - x_{c+1,d-r-c-1}. \end{aligned}$$

而原数阵中，以 $(c, d-r-c-1)$ 为左上位置的三个相邻数满足

$$x_{c,d-r-c-1} + x_{c,d-r-c} + x_{c+1,d-r-c-1} = 0.$$

这些下标均合法，因此

$$y_{r+1,c} = x_{c,d-r-c-1} = x_{c,d-(r+1)-c}.$$

由数学归纳法，该式对整个数阵成立。也就是说，取右边界作为新的第一行后，生成的数阵只是原数阵中各个位置的一次轮换。

再以第二次校准后的序列生成数阵 z。将刚证明的关系应用于 y，可得

$$\begin{aligned} z_{r,c} &= y_{c,d-r-c} \\ &= x_{d-r-c,\, d-c-(d-r-c)} \\ &= x_{d-r-c,r}. \end{aligned}$$

最后，以第三次校准后的序列生成数阵 w，同理有

$$\begin{aligned} w_{r,c} &= z_{c,d-r-c} \\ &= x_{d-c-(d-r-c),\,c} \\ &= x_{r,c}. \end{aligned}$$

因此第三次校准后的第一行满足

$$w_{0,c} = x_{0,c} = a_{c+1} \bmod M,$$

连续三次校准确实还原了初始序列。证明只使用加法与取负，不涉及除法或逆元，所以不要求 M 为素数；$n = 1$ 时同样成立，此时每次校准本身就不会改变序列。

H 星象仪（续）

算法与实现

记一次校准操作为 T，上述结论给出了 $T^3 = I$。若 $k = 3q + r$，其中 $0 \le r < 3$，则

$$T^k = (T^3)^q T^r = T^r.$$

这里不要求每个序列的最小周期都恰好是 3，只需三次操作一定还原即可。

由证明还可以直接读出第二次校准的结果：z 的第一行为

$$z_{0,c} = x_{d-c,0} \qquad (0 \le c \le d),$$

即原数阵的左侧边界从下到上排列。因此，最多只需生成一次完整数阵：

- 若 $k \bmod 3 = 0$，直接输出初始序列各元素对 M 取模后的结果。
- 若 $k \bmod 3 = 1$，输出原数阵每一行最右边的数，顺序为从上到下。
- 若 $k \bmod 3 = 2$，输出原数阵每一行最左边的数，顺序为从下到上。

生成数阵时，不需要保存整个三角形。用一个数组保存当前行，每生成一行，就记录需要的边界值。若当前行长度为 t，可以按 $j = 0, 1, \ldots, t-2$ 的顺序，将位置 j 更新为

$$(-row_j - row_{j+1}) \bmod M,$$

随后将行长减 1。必须从左向右更新，这样计算位置 j 时，它自身和右邻居都还是上一行的值。每次取模后需将负数调整到 0 到 $M-1$ 之间，初始输入中等于 M 的元素也要先归一化为 0。k 应使用 64 位整数保存。

例如样例的右边界为 $(4, 95, 11, 76)$，左边界从下到上为 $(76, 13, 93, 2)$，分别是校准一次和两次的结果；校准三次后回到 $(2, 5, 1, 4)$。由于 $100 \bmod 3 = 1$，校准一百次与校准一次的结果相同。

复杂度

当 $k \bmod 3 = 0$ 时，时间复杂度为 $O(n)$；否则生成数阵需要 $O(n^2)$ 时间。使用当前行数组和结果数组，空间复杂度为 $O(n)$。